

Prisma kézikönyv (magyar)

Ez a kézikönyv a Prisma ORM részletes, gyakorlati útmutatója TypeScript/Node.js környezetben. Tartalmaz példákat schema kezelésre, migrációkra, Prisma Client használatra, relációkra, tranzakciókra, seed-ekre, deploy és CI tippekre, valamint Prisma integrációra NestJS-szel.

Tartalomjegyzék

1. Összefoglaló — Mi a Prisma?
2. Getting started — Telepítés és init
3. Prisma schema — modellek, mezők, típusok
4. Migrations — prisma migrate használata
5. Prisma Client — generálás és használat TypeScript-ben
6. CRUD példák
7. Relációk és lekérdezési minták (1:1, 1:N, N:N, include/select)
8. Transactions és atomic műveletek
9. Indexek, unique constraint-ek, relation modes
10. Prisma Studio, introspect, db push
11. Seed adatok és tesztelés
12. Deploy és CI/CD tippek
13. Performance és hibakeresés (N+1, connection pooling)
14. Security és best practices
15. Prisma + NestJS integráció példa
16. Prisma + GraphQL / REST példák
17. Docker Compose, sample schema és gyakori hibák

1) Összefoglaló — Mi a Prisma?

- A Prisma egy modern ORM és adatbázis eszközkészlet Node.js/TypeScript környezethez. Fő elemei: Prisma schema (schema.prisma), Prisma Client (automatikusan generált, típusbiztos JS/TS kliens), Prisma Migrate (migrációk kezelése) és Prisma Studio (grafikus DB böngésző).
- Előnyök: típusbiztonság TypeScript-ben, tiszta schema definíció, egyszerű migrációs munkafolyamat, jó DX (developer experience).
- Mire jó: CRUD műveletek, komplex relációk kezelése, gyors prototípus és termelési alkalmazások.

2) Getting started — Telepítés és init

1. Telepítsd a Prisma CLI-t és a kliens könyvtárat:

```
npm install prisma --save-dev
npm install @prisma/client
npx prisma init
```

2. A `npx prisma init` létrehoz egy `prisma/schema.prisma` fájlt és egy `.env` fájlt, amelyben a `DATABASE_URL`-t állíthatod be.

3. Példa `.env`:

```
DATABASE_URL="postgresql://user:pass@localhost:5432/mydb?schema=public"
```

3) Prisma schema — modellek, mezők, típusok

Példa schema (Postgres):

```
// prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          Int      @id @default(autoincrement())
  email       String   @unique
  name        String?
  posts       Post[]
  profile     Profile?
  createdAt   DateTime @default(now())
}

model Post {
  id          Int      @id @default(autoincrement())
  title       String
  content     String?
  published   Boolean  @default(false)
  authorId    Int
  author      User     @relation(fields: [authorId], references: [id])
}

model Profile {
  id          Int      @id @default(autoincrement())
```

```

    bio    String?
    userId Int    @unique
    user    User    @relation(fields: [userId], references: [id])
  }

```

Megjegyzések: - `@id`, `@default`, `@unique` és `@relation` direktívák fontosak. - Prisma használ több DB providert: postgresql, mysql, sqlite, sqlserver, mongodb (preview).

4) Migrations — prisma migrate

1. Migrate init és generálás:

```
npx prisma migrate dev --name init
```

- A `migrate dev` fejlesztés közbeni migrációkhoz használatos: módosítja az adatbázist és létrehozza a migration fájlokat.

2. Termelési migrációk:

```
npx prisma migrate deploy
```

- CI/CD pipeline-ban használd, amikor az alkalmazás telepítésekor szeretnéd futtatni a migrációkat.

3. Preview: `prisma migrate reset` (fejlesztéshez: törli adatokat, futtat migrációkat újra).
-

5) Prisma Client — generálás és használat TypeScript-ben

1. Generálás (általában `prisma generate` fut a migrate után):

```
npx prisma generate
```

2. Használat:

```

// src/prisma/client.ts
import { PrismaClient } from '@prisma/client';

export const prisma = new PrismaClient();

```

3. Példa, hogyan használjuk egy service-ben:

```

// src/user.service.ts
import { prisma } from '../prisma/client';

```

```

export async function createUser(data: { name?: string; email: string }) {
  return prisma.user.create({ data });
}

export async function getUsers() {
  return prisma.user.findMany();
}

```

Megjegyzés: PrismaClient példányt általában singleton-ként kell kezelni (különösen dev hot-reload környezetben Next.js/Nodemon esetén ügyelj a lifecycle-re).

6) CRUD példák

Create:

```

const user = await prisma.user.create({
  data: { email: 'alice@example.com', name: 'Alice' },
});

```

Read:

```

const users = await prisma.user.findMany();
const single = await prisma.user.findUnique({ where: { id: 1 } });

```

Update:

```

const updated = await prisma.user.update({
  where: { id: 1 },
  data: { name: 'Alice Updated' },
});

```

Delete:

```

const deleted = await prisma.user.delete({ where: { id: 1 } });

```

7) Relációk és lekérdezési minták

Include és select:

```

// include kapcsolódó posztokat
const userWithPosts = await prisma.user.findUnique({
  where: { id: 1 },
  include: { posts: true },
});

```

```
// select szelektív mezők
const userNameOnly = await prisma.user.findUnique({
  where: { id: 1 },
  select: { id: true, name: true },
});
```

1:N reláció:

```
// létrehozás kapcsolt modellel
const post = await prisma.post.create({
  data: {
    title: 'Hello',
    author: { connect: { id: userId } },
  },
});
```

N:N reláció (példa tags/courses modellek esetén): Prisma generálja a join táblát automatikusan, vagy explicit `@@map/@relation` használható.

8) Transactions és atomic műveletek

Prisma két fő tranzakciós API-t kínál:

- 1) `prisma.$transaction([...queries])` — több lekérdezés egy tranzakcióban:

```
await prisma.$transaction([
  prisma.user.create({ data: { email: 'bob@example.com' } }),
  prisma.post.create({ data: { title: 'tx post', authorId: 1 } }),
]);
```

- 2) Interactive transactions (callback interface) — ajánlott komplex logikához:

```
await prisma.$transaction(async (prismaTx) => {
  const user = await prismaTx.user.create({ data: { email: 'x@y.com' } });
  await prismaTx.post.create({ data: { title: 't', authorId: user.id } });
});
```

Megjegyzés: Ha egy tranzakció több DB kapcsolaton fut (sharding), különös figyelem.

9) Indexek, unique constraint-ek, relation modes

- `@unique` mező szinten biztosítja az egyediségét.
- Kompozit unique és index:

```

model Example {
  a Int
  b Int
  @@unique([a, b])
  @@index([b])
}

```

- `relationMode = "prisma"` vs `"foreignKeys"` (Postgres különböző beállításai). Alap esetben Prisma kezeli a relációk logikáját.
-

10) Prisma Studio, introspect, db push

- Prisma Studio indítása (fejlesztés):

```
npx prisma studio
```

- Adatbázis introspect:

```
npx prisma db pull
```

- `prisma db push` — a schema alapján az adatbázis sémáját tükrözi (nem hoz létre migrációs fájlokat; dev-ben hasznos gyors prototípusra).
-

11) Seed adatok és tesztelés

Seed script (package.json):

```

{
  "prisma": {
    "seed": "ts-node prisma/seed.ts"
  }
}

```

prisma/seed.ts vázlat:

```

import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();

async function main() {
  await prisma.user.create({
    data: { email: 'seed@example.com', name: 'Seed' },
  });
}

main()

```

```
.catch(e => console.error(e))  
.finally(async () => { await prisma.$disconnect(); });
```

Tesztelés: - Teszt környezethez használj külön adatbázist (CI-ben ephemeral DB).
- Mockolás helyett inkább integration teszteket futtass valódi DB-vel (SQLite in-memory vagy Dockerized Postgres).

12) Deploy és CI/CD tippek

- CI pipeline:
 1. build
 2. `npx prisma migrate deploy` (termelési migrációk)
 3. `npm run start`
 - Használj environment-specifikus `DATABASE_URL`-t.
 - Biztonság: soha ne tárold a DB jelszavát a forráskódban, használj secret manager-t.
-

13) Performance és hibakeresés

N+1 probléma: - GraphQL + Prisma esetén gyakori. Megoldás: DataLoader vagy optimalizált include/select stratégiák.

Connection pooling: - Prisma Client Pooling használata a DB driver rétegével (pl. PgBouncer Postgres esetén). - Figyeld a max connections limitet termelésben.

Lekérdezés optimalizálás: - Csak a szükséges mezőket kérd (`select`), használd az `include`-t tudatosan. - Használj indexeket gyakran szűrt mezőkhöz.

14) Security és best practices

- Least privilege: alkalmazás DB user legyen limitált jogosultságokkal (csak szükséges sémák/olvasás-írás).
 - Secrets: használj Vault/AWS Secrets Manager/GitHub Secrets.
 - Input validation: Prisma önmagában nem validál minden logikát — használj szerver oldali validációt (class-validator stb.).
 - SQL injection: Prisma kliens biztonságos paraméterezésen alapul, de ügyelj raw query-k használatakor.
-

15) Prisma + NestJS integráció példa

- Telepítés:

```
npm i @prisma/client prisma
npm i -D prisma
```

- PrismaService példány (singleton, lifecycle kezeléssel):

```
// src/prisma/prisma.service.ts
import { Injectable, OnModuleInit, OnModuleDestroy } from '@nestjs/common';
import { PrismaClient } from '@prisma/client';

@Injectable()
export class PrismaService extends PrismaClient implements OnModuleInit, OnModuleDestroy {
  async onModuleInit() {
    await this.$connect();
  }
  async onModuleDestroy() {
    await this.$disconnect();
  }

  // Optional: add enableShutdownHooks for Nest
  async enableShutdownHooks(app: any) {
    this.$on('beforeExit', async () => {
      await app.close();
    });
  }
}
```

- PrismaModule:

```
// src/prisma/prisma.module.ts
import { Global, Module } from '@nestjs/common';
import { PrismaService } from './prisma.service';

@Global()
@Module({
  providers: [PrismaService],
  exports: [PrismaService],
})
export class PrismaModule {}
```

- Használat egy service-ben:

```
@Injectable()
export class UsersService {
  constructor(private prisma: PrismaService) {}
```

```

    findAll() {
      return this.prisma.user.findMany();
    }
  }
}

```

16) Prisma + GraphQL / REST minták

GraphQL (resolver-ben):

```

@Resolver(() => UserType)
export class UsersResolver {
  constructor(private prisma: PrismaService) {}

  @Query(() => [UserType])
  users() {
    return this.prisma.user.findMany({ include: { posts: true }});
  }
}

```

REST (controller/service):

```

@Get()
async getUsers() {
  return this.usersService.findAll();
}

```

17) Docker Compose példa (Postgres + Prisma)

```

version: '3.8'
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mydb
    ports:
      - "5432:5432"
    volumes:
      - dbdata:/var/lib/postgresql/data

  app:
    build: .

```

```
environment:
  DATABASE_URL: "postgresql://user:pass@db:5432/mydb?schema=public"
depends_on:
  - db

volumes:
  dbdata:
```

Gyakori hibák és megoldások

- “PrismaClientKnownRequestError” — ellenőrizd constraint-eket és migrációkat.
 - Hot-reload alatt többszörös PrismaClient példány — singleton kezelése szükséges (pl. egyetlen PrismaService).
 - Migrációs konfliktusok több fejlesztőnél — merge előtt futtasd a migrációk tesztjét.
-

Források és további olvasmányok

- Prisma docs: <https://www.prisma.io/docs>
 - Prisma GitHub repo: <https://github.com/prisma/prisma>
-

(Kérésre elkészítem a PDF-et is és segítek feltölteni a repódba vagy külső hostra. Ha szeretnéd, összeállítom ugyanazt a workflow-t, mint a NestJS esetén, hogy a Markdownból automatikusan PDF készüljön.)